Лабораторная работа №2

Введение в операционные системы. Архитектура компьютеров и операционные системы

Лебедев С. А.

Содержание

1 Цель работы **5**

2 Задание **6**

3 Теоретическое введение **7**

4 Выполнение лабораторной работы **8**

4.0.1 Установка программного обеспечения 8
4.0.2 Базовая настройка git 8
4.0.3 Создание ключей SSH и PGP 9
4.0.4 Добавление PGP ключа в GitHub и настройка подписи коммитов 10
4.0.5 Настройка gh (GitHub CLI) 10
4.0.6 Создание репозитория курса на основе шаблона 10
4.0.7 Настройка каталога курса и отправка файлов на сервер . . . 11

5 Ответы на контрольные вопросы **12**

6 Выводы **15**

Список литературы **16**

Список иллюстраций

4.1 Установка git и gh 8
4.2 Базовая настройка git 9
4.3 Создание SSH ключа 9
4.4 Создание PGP (GPG) ключа 9
4.5 Настройка подписи коммитов и добавление ключа 10
4.6 Авторизация gh 10
4.7 Клонирование репозитория 11
4.8 Отправка изменений на сервер 11

Список таблиц

1 Цель работы

Целью лабораторной работы является изучение идеологии и применения средств контроля версий, а также освоение практических навыков работы с системой контроля версий **Git** (установка, первичная настройка, создание SSH и PGP ключей, настройка подписи коммитов и подготовка рабочего репозитория).

2 Задание

1. Создать базовую конфигурацию для работы с `git`.
2. Создать ключ SSH и добавить его в учётную запись GitHub.
3. Создать ключ PGP (GPG) и добавить его в учётную запись GitHub.
4. Настроить автоматическую подпись коммитов в `git`.
5. Авторизоваться в GitHub с помощью `gh` (GitHub CLI).
6. Создать локальный каталог и репозиторий курса на основе шаблона, отправить изменения на сервер.

3 Теоретическое введение

Система контроля версий (VCS) — это инструмент для хранения истории изменений проекта и организации совместной работы. VCS позволяет фиксировать изменения (коммиты), возвращаться к предыдущим версиям, объединять изменения разных участников и отслеживать авторство правок.

Git является распределённой системой контроля версий: у каждого участника хранится полная копия репозитория и история изменений. Для безопасной работы с удалёнными репозиториями на GitHub используются:

- **SSH-ключи** — для аутентификации при подключении к репозиторию;
- **PGP/GPG-подписи** — для подтверждения авторства коммитов (на GitHub отображается статус *Verified*).

4 Выполнение лабораторной работы

4.0.1 Установка программного обеспечения

Выполнена установка пакетов `git` и `gh` в операционной системе. Результат установки представлен на рис. 4.1.

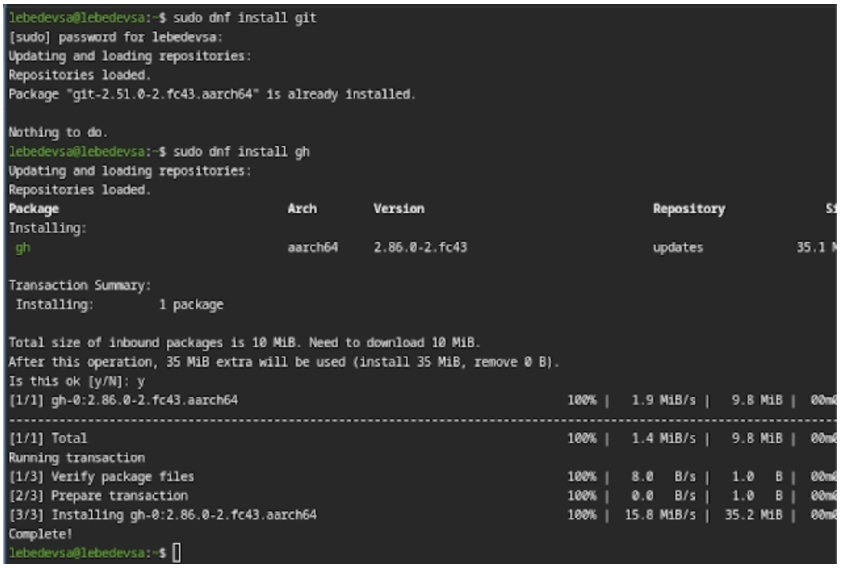

Рис. 4.1: Установка git и gh

4.0.2 Базовая настройка git

Выполнена первичная настройка `git`: задано имя пользователя и email, настроен корректный вывод путей (UTF-8), параметры учёта переносов строк (`autocrlf`, `safecrlf`) и необходимые базовые параметры. Результат настройки представлен на рис. 4.2.

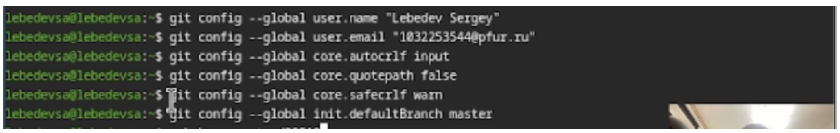

Рис. 4.2: Базовая настройка git

4.0.3 Создание ключей SSH и PGP

Сгенерирован SSH-ключ (RSA 4096 бит) и создан PGP (GPG) ключ для подписи коммитов. Результат генерации ключей представлен на рис. 4.3 и рис. 4.4.

Рис. 4.3: Создание SSH ключа

Рис. 4.4: Создание PGP (GPG) ключа

4.0.4 Добавление PGP ключа в GitHub и настройка подписи коммитов

Выполнен экспорт публичного PGP ключа и добавление ключа в учётную запись GitHub. Затем настроена автоматическая подпись коммитов: установлен `user.signingkey`, включён параметр `commit.gpgsign`. Результат представлен на рис. 4.5.

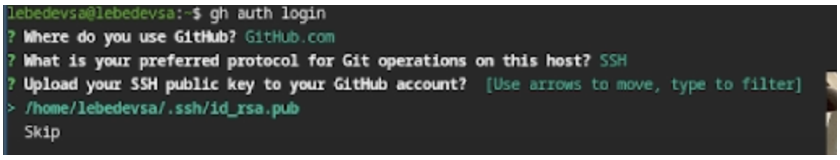

Рис. 4.5: Настройка подписи коммитов и добавление ключа

4.0.5 Настройка gh (GitHub CLI)

Выполнена авторизация в gh для работы с GitHub через терминал. Результат авторизации представлен на рис. 4.6.

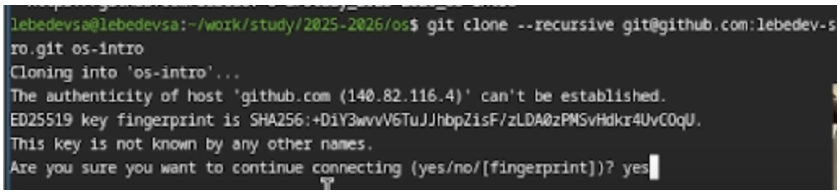

Рис. 4.6: Авторизация gh

4.0.6 Создание репозитория курса на основе шаблона

Создан репозиторий курса на основе шаблона и выполнено клонирование репозитория на локальную машину. Результат представлен на рис. 4.7.

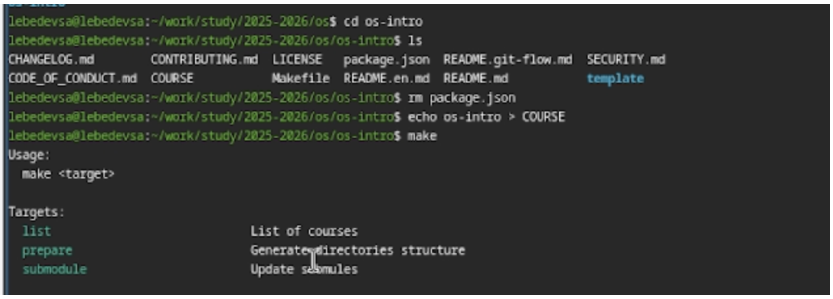

Рис. 4.7: Клонирование репозитория

4.0.7 Настройка каталога курса и отправка файлов на сервер

Удалены лишние файлы, создан файл `COURSE`, сформирована структура каталога курса, затем изменения отправлены в удалённый репозиторий. Результат отправки представлен на рис. 4.8.

Рис. 4.8: Отправка изменений на сервер

5 Ответы на контрольные вопросы

1. Что такое системы контроля версий (VCS) и для каких задач они предназначены? VCS — система контроля версий, предназначенная для хранения истории изменений проекта. Она позволяет фиксировать изменения, возвращаться к предыдущим версиям, организовывать совместную работу, отслеживать авторство правок и решать конфликты при одновременном редактировании.

2. Понятия: хранилище, commit, история, рабочая копия * **Хранилище (repository)** — место хранения проекта и его истории изменений. * **Commit** — фиксированное изменение (снимок состояния проекта) с описанием. * **История** — последовательность коммитов, отражающая развитие проекта. * **Рабочая копия** — текущие файлы на компьютере пользователя, которые он редактирует.

3. Чем отличаются централизованные и распределённые VCS? Примеры Централизованные VCS имеют единый сервер (например, CVS, Subversion/SVN). Распределённые VCS хранят полную историю у каждого участника (например, Git, Mercurial, Bazaar). Главное отличие — возможность полноценной работы локально без постоянного подключения к серверу.

4. Действия с VCS при единоличной работе с хранилищем Создание репозитория (`git init`), внесение изменений в файлы, подготовка изменений (`git add`), фиксация (`git commit`), просмотр состояния и истории (`git status`, `git log`), при необходимости — откат к нужной версии.

5. Порядок работы с общим хранилищем VCS Получить актуальные изменения (`git pull`), внести правки, добавить изменения (`git add`), зафиксировать (`git commit`), отправить в удалённый репозиторий (`git push`). При возникнове-

нии конфликтов выполнить слияние и разрешить конфликты.

6. Основные задачи, решаемые git Хранение истории изменений, контроль версий, ветвление и слияние, совместная разработка, синхронизация с удалёнными репозиториями, восстановление состояния проекта.

7. Краткая характеристика основных команд git * `git init` — создать локальный репозиторий * `git status` — показать состояние файлов * `git add` — добавить изменения в индекс * `git commit` — зафиксировать изменения * `git log` — показать историю коммитов * `git diff` — показать отличия * `git pull` — получить изменения с сервера * `git push` — отправить изменения на сервер * `git branch` — работа с ветками * `git checkout` — переключение веток/состояний * `git merge` — слияние веток

8. Примеры работы с локальным и удалённым репозиториями

Локальная работа:

```
git init
git add .
git commit -m "Initial commit"
```

9. Что такое ветви (branches) и зачем они нужны? Ветви - это независимые линии разработки внутри одного репозитория. Они позволяют: • разрабатывать новые функции отдельно от основной версии, • тестировать изменения без риска сломать проект, • вести параллельную разработку, • объединять изменения с помощью merge.

```
git checkout -b new-feature
git merge new-feature
```

10. Как и зачем игнорировать файлы при commit?

Файл .gitignore используется для исключения файлов и каталогов из отслеживания Git. Это необходимо для: • временных файлов, • логов, • скомпилированных файлов, • системных и служебных файлов.

Пример .gitignore:

```
*.log
node_modules/
__pycache__/
build/
*.tmp
```

6 Выводы

В ходе выполнения лабораторной работы были установлены git и gh, выполнена первичная настройка Git, созданы SSH и PGP (GPG) ключи, настроена автоматическая подпись коммитов, выполнена авторизация через GitHub CLI, создан репозиторий курса на основе шаблона и отправлены изменения на сервер. Таким образом, получены практические навыки настройки и работы с Git в связке с GitHub.

Список литературы